

LANE KEEPING CONTROL

Detección de Carriles y Control de Mantenimiento de Carril

Visión Artificial · Control PID · ROS 2 · CARLA Simulator

José López · Rafael Muñoz · Agustín Mayor

Python

OpenCV

ROS 2

CARLA

MATLAB

Índice

01

Motivación y Objetivo

Contexto ADAS · Sistema LKA

02

Detección de Carriles

Pre-procesamiento · OpenCV · Python

03

Control PID

Sintonización · Ganancias · Bucle cerrado

04

Simulación MATLAB

Validación teórica en Simulink

05

ROS 2 + CARLA

Implementación en simulador realista

06

Resultados y Conclusiones

Validación del sistema completo

Motivación y Objetivo

¿Por qué?

Los sistemas ADAS son clave en la conducción autónoma moderna. El Lane Keeping Assist es una función fundamental.

¿Qué hace?

Detecta el carril mediante cámara y corrige la trayectoria del vehículo en tiempo real para mantenerlo centrado.

¿Cómo?

Integra visión artificial (OpenCV), control PID y simulación en un pipeline de bucle cerrado completo.

OBJETIVO

Desarrollar un sistema completo de mantenimiento de carril: desde la imagen de cámara hasta la consigna de giro del volante, validado en simulación.

Cámara



Visión



Error



PID



Volante

Detección de Carriles

Visión Artificial con OpenCV · Python

1

Escala de grises

Convierte la imagen a 1 canal para simplificar el procesamiento

2

Filtro Gaussiano

Elimina ruido antes de aplicar el detector de bordes

3

Detector Canny

Localiza bordes mediante gradiente de intensidad

4

Hough Transform

Detecta segmentos de línea recta en la imagen

5

Ajuste de carril

Separa líneas izq/dcha y calcula el centro del carril

Resultado: Centro del carril = punto medio entre línea izq. y dcha. → Error lateral (m) → entrada al PID.

Controlador PID

Control en cascada · Lane Keeping

K_p

0.00200

Proporcional
Responde al error instantáneo

K_i

0.00100

Integral
Elimina el error estacionario

K_d

-1.07×10^{-5}

Derivativo
Amortigua oscilaciones

Error lateral



PID



Ángulo volante



Vehículo



Cámara



Vehículo de referencia: Tesla — ganancias sintonizadas en MATLAB/Simulink y transferidas directamente a Python/ROS 2

MATLAB / Simulink

Validación teórica del control

Modelo del vehículo

Modelo dinámico del Tesla en Simulink con parámetros reales

Error artificial

Error lateral introducido manualmente para probar la respuesta del PID

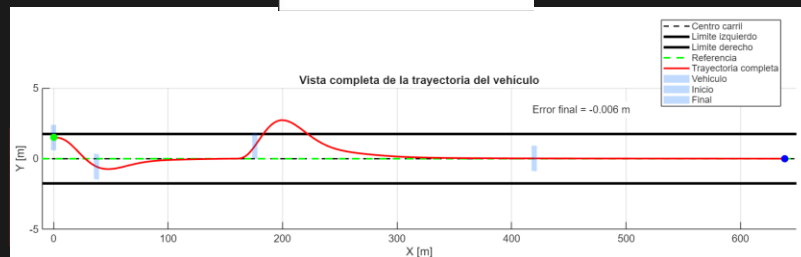
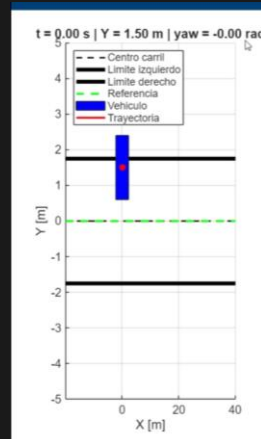
Salida del modelo

Ángulo de giro del volante como consigna al actuador

Resultado

Error final = -0.006 m · Convergencia al centro del carril

Vista completa de la trayectoria — Simulink



CARLA — Simulador

ROS 2

Carla ROS Bridge

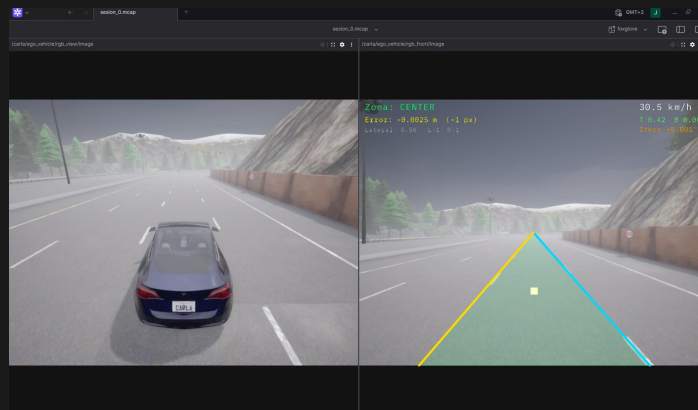
SUB /rgb_front/image

SUB /speedometer

PUB /vehicle_control_cmd

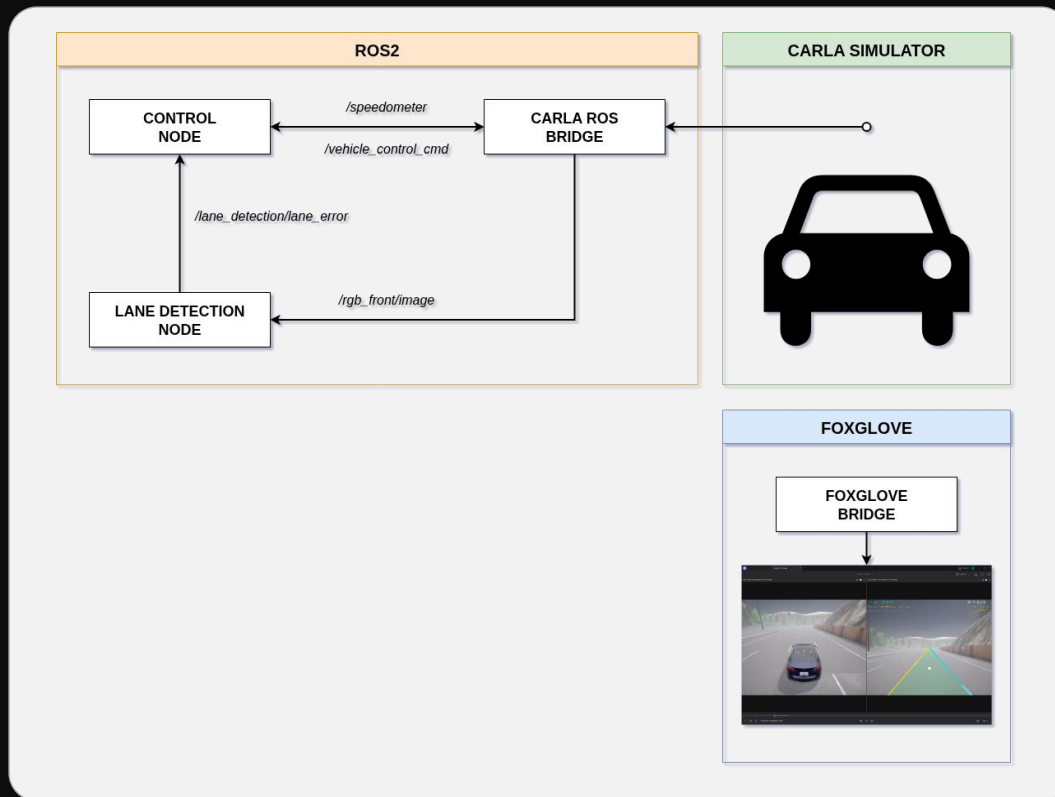
MSG CarlaEgoVehicleControl

CARLA Simulator 0.9.15
Detección en tiempo real

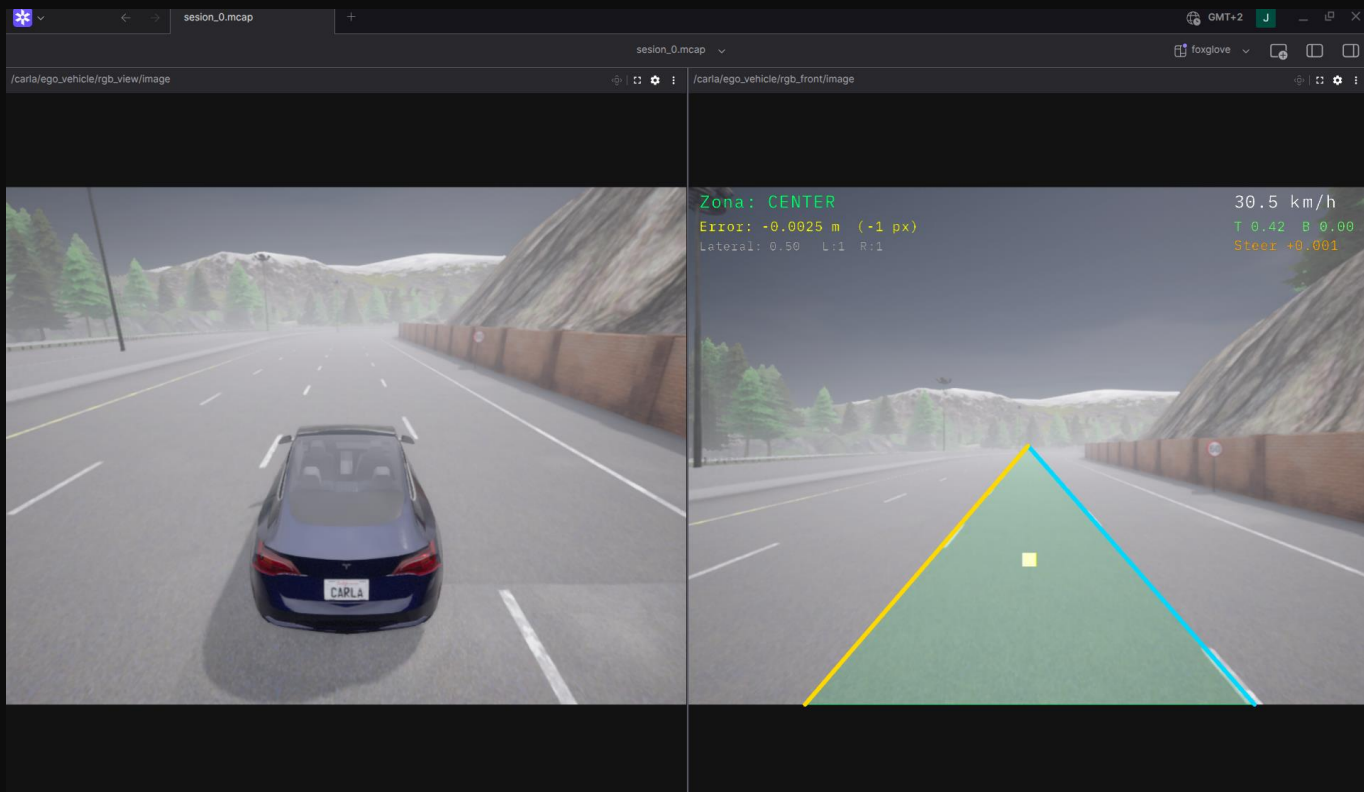


Izquierda: vista exterior del vehículo · Derecha: cámara frontal con líneas de carril detectadas (naranja/azul) y error lateral en tiempo real

ROS 2 — Arquitectura



Foxglove — Interfaz



Resultados



El sistema detecta correctamente el centro del carril a partir de imágenes de la cámara frontal



El controlador PID corrige el error lateral y mantiene el vehículo centrado en el carril



Las ganancias PID sintonizadas en Simulink funcionaron sin ajuste adicional en Python/ROS 2

MATLAB / Simulink

Respuesta correcta ante errores laterales artificiales. Sintonización del PID completada.

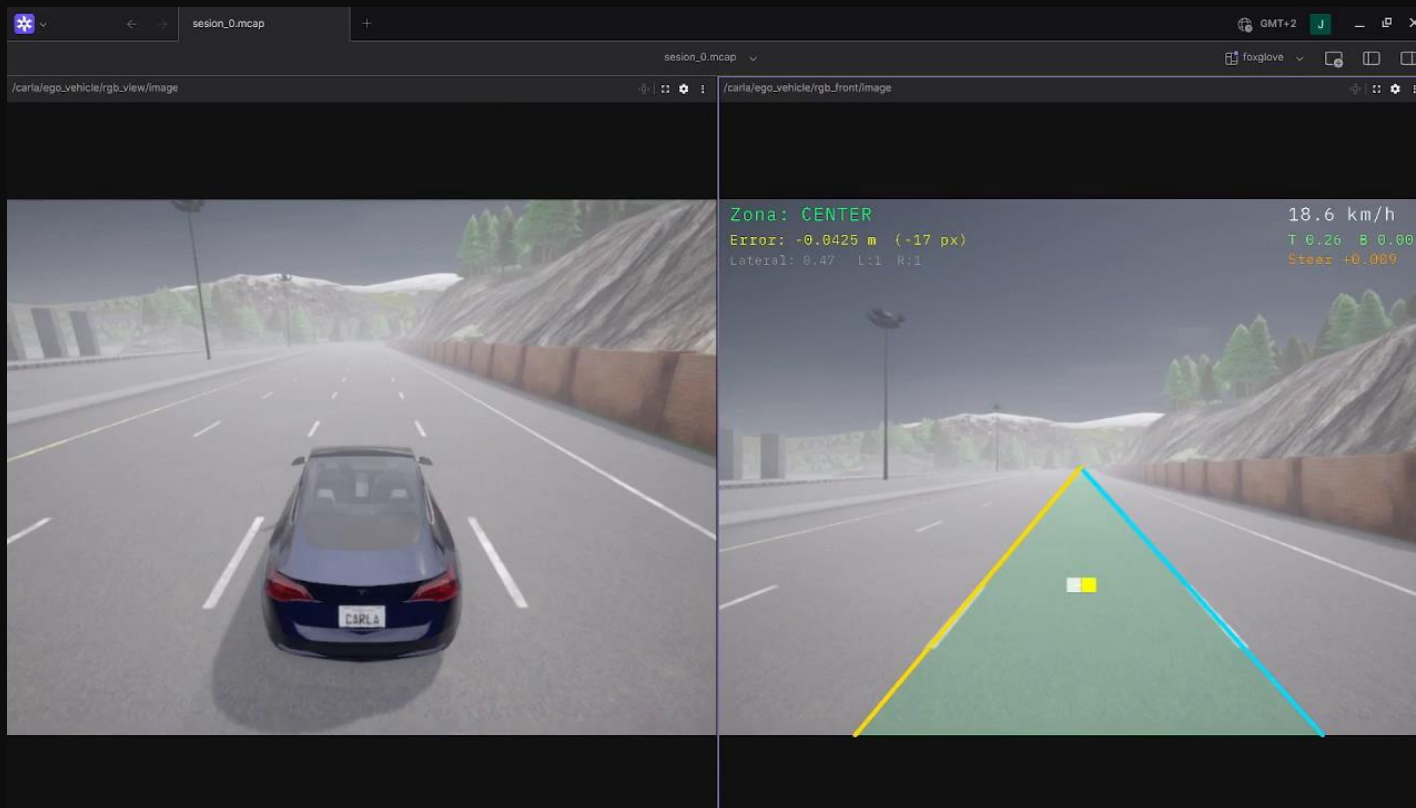
ROS 2 + CARLA

Pipeline completo operativo en simulación realista con vehículo Tesla virtual.

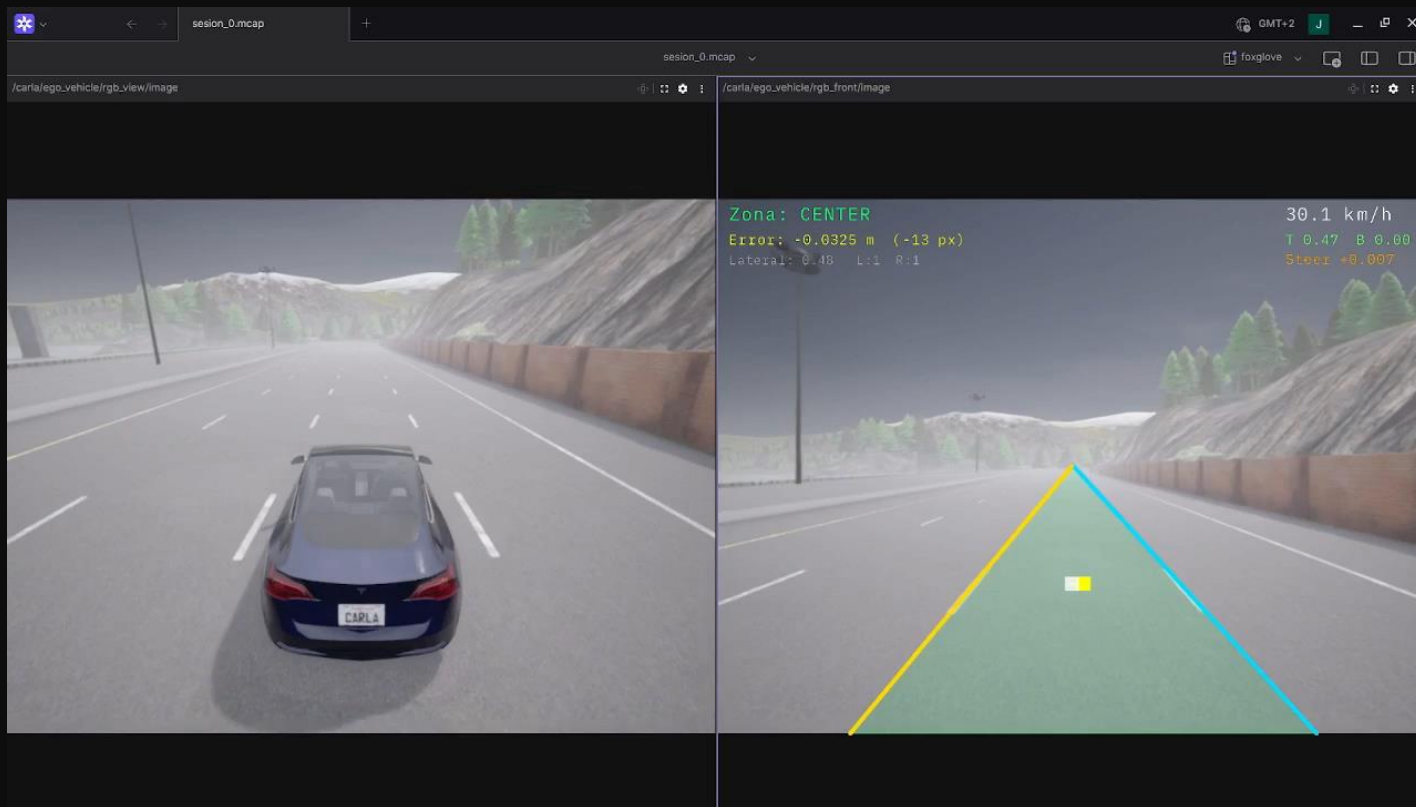
Demo — Detección de Carriles



Demo — Simulador (18 km/h)



Demo — Simulador (30 km/h)



Conclusiones

01

Se ha implementado con éxito un sistema de lane keeping de extremo a extremo: desde la imagen hasta el actuador.

02

La combinación de visión artificial con OpenCV y control PID es efectiva y transferible a entornos reales.

03

El uso de ROS 2 facilita la modularidad del sistema y la integración con simuladores de conducción autónoma.

04

El trabajo cubre los tres pilares del vehículo autónomo: percepción, planificación y control.

05

Herramientas estándar en la industria: OpenCV, ROS 2, CARLA y MATLAB/Simulink.

Fin

Tecnologías Utilizadas

Python

Implementación del detector y controlador

OpenCV

Procesamiento de imagen y detección de carriles

ROS 2

Comunicación entre nodos del sistema

CARLA Simulator

Simulador de conducción autónoma realista

MATLAB/Simulink

Modelado teórico y sintonización PID

GitHub

Control de versiones y colaboración del equipo